Платформено-независими програмни езици

Лекция 8

Нишки и мрежово програмиране с TCP

Конкурентност, синхронизация, виртуални нишки, сокети, клиент-сървър и HTTP

8

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.

Съдържание на лекцията

1. Процеси и нишки
2. Създаване и управление на нишки
3. Синхронизация
4. Изпълнители и асинхронни задачи
5. Основи на компютърните мрежи
6. TCP програмиране в Java
7. HTTP – протокол върху TCP

01

Процеси и нишки

Защо програмите правят няколко неща едновременно

Процес и нишка

Процес

- самостоятелна изпълнявана програма
- собствена памет
- изолиран от другите процеси
- скъпо създаване и превключване
- пример: всяко стартирано `java` приложение

Нишка (thread)

- път на изпълнение **вътре** в процеса
- споделя паметта с другите нишки на процеса
- по-евтина от процес
- комуникира чрез общи обекти
- пример: сървър, обслужващ много клиенти

Защо са нужни нишки?

- **Отзивчивост** – програмата реагира, докато върши дълга работа
- **Сървъри** – едновременно обслужване на много клиенти
- **Изчакване на вход/изход** – докато една нишка чака мрежата, друга работи
- **Използване на многоядрени процесори** – паралелни изчисления
- Цената: нова категория грешки – **състезания**, **блокировки**, трудно възпроизводими бъгове

Конкурентност и паралелизъм

Конкурентност (concurrency)

- няколко задачи **напредват** в едно и също време
- могат да се редуват на едно ядро
- въпрос на **структура** на програмата
- пример: сървър с хиляди връзки

Паралелизъм (parallelism)

- задачи се изпълняват **буквално едновременно**
- изисква няколко ядра
- въпрос на **производителност**
- пример: обработка на голям масив на 8 ядра

Жизнен цикъл на нишка

Състояние	Значение
`NEW`	създадена, но не е стартирана
`RUNNABLE`	изпълнява се или е готова за изпълнение
`BLOCKED`	чака да получи заключване (`synchronized`)
`WAITING`	чака друга нишка неограничено (`join`, `wait`)
`TIMED_WAITING`	чака за определено време (`sleep`)
`TERMINATED`	приключила

02

Създаване и управление на нишки

Thread, Runnable, join, виртуални нишки

Създаване на нишка

- Задачата – обект `Runnable` (често ламбда)
- `start()` – стартира нова нишка, която изпълнява `run()`
- **Не** извиквайте `run()` директно – ще се изпълни в текущата нишка
- Редът на изпълнение между нишките **не е определен**

```
Runnable task = () -> {
    String me = Thread.currentThread().getName();
    for (int i = 1; i <= 3; i++) {
        System.out.println(me + ": " + i);
    }
};

Thread t1 = new Thread(task, "A");
Thread t2 = new Thread(task, "B");
t1.start();
t2.start();
System.out.println("main продължава");
// редът на изхода е различен при всяко пускане!
```

sleep и join

- `Thread.sleep(ms)` – приспива текущата нишка
- `join()` – изчаква друга нишка да приключи
- И двата хвърлят `InterruptedException`

```
Thread worker = new Thread(() -> {
    try {
        System.out.println("Работя...");
        Thread.sleep(2000);
        System.out.println("Готово");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});

worker.start();
worker.join();      // main чака тук
System.out.println("main: работникът приключи");
```

Прекъсване на нишка

- Нишка не се „убива“ отвън – **моли се** да спре
- `interrupt()` – вдига флаг на нишката
- Нишката проверява `isInterrupted()` и спира сама
- `sleep`/`wait` хвърлят `InterruptedException`

```
Thread ticker = new Thread(() -> {
    int n = 0;
    while (!Thread.currentThread()
                  .isInterrupted()) {
        System.out.println("тик " + n++);
        try {
            Thread.sleep(500);
        } catch (InterruptedException e) {
            break;     // излизаме чисто
        }
    }
    System.out.println("спрях");
});
ticker.start();
Thread.sleep(2000);
ticker.interrupt();
```

Виртуални нишки (Java 21)

- Класическите нишки са **нишки на ОС** – скъпи; хиляди, не милиони
- **Виртуалните нишки** се управляват от JVM – изключително леки
- Когато виртуална нишка чака вход/изход, ядрото се освобождава за друга
- Позволяват стил „**една нишка на заявка**“ дори при стотици хиляди връзки
- Идеални за сървъри и мрежови приложения
- Не правят изчисленията по-бързи – помагат при **чакане**

Виртуални нишки – пример

- `Thread.ofVirtual().start(...)`
- `Thread.startVirtualThread(...)`
- Изпълнител с по една виртуална нишка на задача
- 10 000 задачи по 1 секунда → около 1 секунда общо

```
Thread v = Thread.ofVirtual()
        .start(() -> System.out.println(
                "Виртуална нишка!"));
v.join();

try (var ex = Executors
        .newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        int id = i;
        ex.submit(() -> {
            Thread.sleep(1000);   // „вход/изход“
            return id;
        });
    }
}   // close() изчаква всички задачи
```

Нишки в Python и в Java

Python (CPython)

- модул `threading`
- **GIL** – само една нишка изпълнява Python код в даден момент
- нишките помагат при чакане на вход/изход
- за изчисления – `multiprocessing` (процеси)
- `asyncio` – асинхронен стил

Java

- `Thread`, `ExecutorService`, виртуални нишки
- **няма GIL** – нишките работят истински паралелно
- помагат и при вход/изход, и при изчисления
- затова синхронизацията е още по-важна
- `CompletableFuture` – асинхронен стил

03

Синхронизация

Споделени данни и състезания между нишки

Състезание (race condition)

```
class Counter {
    private int value = 0;
    void increment() { value++; }      // НЕ е атомарна операция!
    int get() { return value; }
}

Counter c = new Counter();
Runnable r = () -> { for (int i = 0; i < 100_000; i++) c.increment(); };

Thread t1 = new Thread(r), t2 = new Thread(r);
t1.start(); t2.start();
t1.join();  t2.join();

System.out.println(c.get());   // очакваме 200000, получаваме напр. 137452
```

`value++` = прочети, добави 1, запиши. Двете нишки се „застъпват“ и губят увеличения.

synchronized

- Само **една** нишка може да изпълнява синхронизиран код на даден обект
- Останалите чакат (състояние `BLOCKED`)
- Синхронизиран метод или блок
- Гарантира и **видимост** на промените

```
class Counter {
    private int value = 0;

    synchronized void increment() {
        value++;
    }

    synchronized int get() {
        return value;
    }
}

// блок върху конкретен обект
synchronized (lock) {
    total += x;
}
```

Атомарни променливи

- `java.util.concurrent.atomic`
- `AtomicInteger`, `AtomicLong`, `AtomicReference`
- Атомарни операции без заключване – по-бързи
- `LongAdder` – за много нишки, които само броят

```
class Counter {
    private final AtomicInteger value =
            new AtomicInteger();

    void increment() {
        value.incrementAndGet();
    }

    int get() {
        return value.get();
    }
}

AtomicInteger max = new AtomicInteger();
max.updateAndGet(m -> Math.max(m, 42));
```

Конкурентни колекции

Обикновена	Безопасна за нишки	Бележка
`HashMap`	`ConcurrentHashMap`	без заключване на цялата таблица
`ArrayList`	`CopyOnWriteArrayList`	за много четене, рядък запис
`ArrayDeque`	`ConcurrentLinkedQueue`	неблокираща опашка
–	`ArrayBlockingQueue`, `LinkedBlockingQueue`	производител–потребител
`HashSet`	`ConcurrentHashMap.newKeySet()`	конкурентно множество

Производител – потребител

- `BlockingQueue` – `put` чака при пълна опашка, `take` – при празна
- Класически модел за разпределяне на работа между нишки
- Синхронизацията е скрита в опашката

```
BlockingQueue<String> q =
        new ArrayBlockingQueue<>(10);

Thread.startVirtualThread(() -> {
    try {
        for (int i = 1; i <= 5; i++)
            q.put("задача " + i);
    } catch (InterruptedException e) { }
});

Thread.startVirtualThread(() -> {
    try {
        while (true)
            System.out.println(q.take());
    } catch (InterruptedException e) { }
});
```

CountDownLatch – изчакване на няколко задачи

- Брояч, който нишките намаляват с `countDown()`
- `await()` чака броячът да стане 0
- Пример: стартираме услуга, когато всички части са готови

```
CountDownLatch ready = new CountDownLatch(3);

for (String part : List.of("база", "кеш",
                           "конфигурация")) {
    Thread.startVirtualThread(() -> {
        System.out.println("Зареждам " + part);
        ready.countDown();
    });
}

ready.await();          // чака и трите
System.out.println("Услугата е готова");
```

Взаимно блокиране (deadlock)

- Нишка A държи ресурс 1 и чака ресурс 2
- Нишка B държи ресурс 2 и чака ресурс 1
- И двете чакат вечно – програмата „замръзва“
- Как да се избегне:
  - заключвайте ресурсите винаги в **един и същ ред**
  - дръжте заключването възможно най-кратко
  - използвайте `tryLock` с време за изчакване
  - предпочитайте готови конкурентни структури



Най-безопасните споделени данни са неизменяемите

Неизменяем обект (`record`, `String`, `List.of`) може да се чете от колкото и да е нишки без синхронизация. Споделяйте неизменяеми данни, а изменяемите – дръжте локални за нишката.

04

Изпълнители и асинхронни задачи

ExecutorService, Callable, Future, CompletableFuture

Защо изпълнители (executors)?

- Ръчното създаване на нишки е трудно за управление
- **ExecutorService** – пул от нишки, които изпълняват подадени задачи
- Разделяме **какво** се изпълнява (задача) от **как** (нишки)
- Видове:
  - `newFixedThreadPool(n)` – фиксиран брой нишки, за изчисления
  - `newVirtualThreadPerTaskExecutor()` – за вход/изход
  - `newSingleThreadExecutor()` – последователно изпълнение

Callable и Future

- `Callable<T>` – задача, която **връща** резултат и може да хвърли изключение
- `submit` връща `Future<T>` – „обещание“ за резултат
- `get()` – изчаква резултата

```
ExecutorService pool =
        Executors.newFixedThreadPool(4);

Callable<Long> task = () ->
        LongStream.rangeClosed(1, 50_000_000)
                  .sum();

Future<Long> f = pool.submit(task);
System.out.println("Изчислявам...");
long sum = f.get();     // чака
System.out.println(sum);

pool.shutdown();
```

CompletableFuture

- Асинхронна верига от стъпки – без блокиране
- `supplyAsync` – стартира задача
- `thenApply`, `thenCombine`, `thenAccept` – следващи стъпки
- `exceptionally` – обработка на грешки

```
CompletableFuture<Double> price =
    CompletableFuture.supplyAsync(
            () -> fetchPrice("BTC"));

CompletableFuture<Double> rate =
    CompletableFuture.supplyAsync(
            () -> fetchRate("EUR"));

price.thenCombine(rate, (p, r) -> p * r)
     .thenApply(v -> "%.2f €".formatted(v))
     .exceptionally(e -> "няма данни")
     .thenAccept(System.out::println)
     .join();
```

Паралелни HTTP заявки

- Всяка заявка – в своя виртуална нишка
- Общото време ≈ времето на най-бавната заявка
- `invokeAll` – подава списък от задачи

```
var client = HttpClient.newHttpClient();
List<String> urls = List.of(
        "https://example.com",
        "https://example.org");

try (var ex = Executors
        .newVirtualThreadPerTaskExecutor()) {
    var tasks = urls.stream()
        .map(u -> (Callable<Integer>) () ->
            client.send(HttpRequest.newBuilder(
                    URI.create(u)).build(),
                HttpResponse.BodyHandlers.ofString())
              .statusCode())
        .toList();
    for (var f : ex.invokeAll(tasks))
        System.out.println(f.get());
}
```

05

Основи на компютърните мрежи

IP адреси, портове, TCP и UDP

Модел TCP/IP

Слой	Предназначение	Примери
Приложен	данни на приложенията	HTTP, SMTP, DNS, собствен протокол
Транспортен	връзка между програми, портове	**TCP**, UDP
Мрежов	адресиране и маршрутизиране между мрежи	IP (IPv4, IPv6)
Канален / физически	предаване в локалната мрежа	Ethernet, Wi-Fi

IP адреси и портове

- **IP адрес** – идентифицира компютър в мрежата
  - IPv4: `192.168.1.10`, IPv6: `2001:db8::1`
  - `127.0.0.1` / `localhost` – текущият компютър
- **DNS** превежда имена в адреси: `dev.java` → IP
- **Порт** (0–65535) – идентифицира **програма** на компютъра
  - 80 – HTTP, 443 – HTTPS, 22 – SSH, 5432 – PostgreSQL
  - портове под 1024 изискват администраторски права
- Връзка = (IP на клиента, порт) ↔ (IP на сървъра, порт)

TCP и UDP

TCP

- установява **връзка** преди обмен
- гарантирана доставка и **ред** на данните
- поток от байтове
- по-бавен, повече служебни данни
- HTTP, имейл, SSH, бази данни

UDP

- **без** връзка – отделни пакети (датаграми)
- без гаранция за доставка и ред
- по-бърз, по-малко служебни данни
- подходящ при загуба на данни без значение
- DNS, видео, онлайн игри

Модел клиент – сървър

- **Сървър** – стартира първи, **слуша** на известен порт, чака връзки
- **Клиент** – знае адреса и порта на сървъра, **инициира** връзката
- След установяване – двупосочен поток от данни
- **Протокол** – договорени правила: кой какво изпраща и в какъв формат
- Един сървър обслужва много клиенти → нишки
- Сокет (socket) – крайна точка на връзката: IP + порт

InetAddress – имена и адреси

- `InetAddress` представя IP адрес
- `getByName` – DNS заявка
- `getLocalHost` – текущият компютър

```
InetAddress a =
        InetAddress.getByName("dev.java");
System.out.println(a.getHostAddress());

InetAddress[] all =
        InetAddress.getAllByName("google.com");
for (InetAddress x : all) {
    System.out.println(x);
}

InetAddress me = InetAddress.getLocalHost();
System.out.println(me.getHostName());

InetAddress lo =
        InetAddress.getLoopbackAddress();
// localhost/127.0.0.1
```

06

TCP програмиране в Java

Socket и ServerSocket

Класовете за TCP

Клас	Роля	Основни методи
`ServerSocket`	сървър: слуша на порт	`accept()` – чака и приема клиент
`Socket`	двата края на връзката	`getInputStream()`, `getOutputStream()`, `close()`
`BufferedReader`	четене на редове текст от сокета	`readLine()`
`PrintWriter`	запис на редове текст в сокета	`println()`, `flush()`

Ехо сървър: приемане на клиенти

```
public class EchoServer {

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(5000)) {
            System.out.println("Слушам на порт 5000...");
            while (true) {
                Socket client = server.accept();     // чака клиент
                System.out.println("Клиент: " + client.getRemoteSocketAddress());
                handle(client);                      // обслужва го
            }
        }
    }
}
```

`accept()` блокира, докато не се свърже клиент, и връща сокет за връзката с него

Ехо сървър: обслужване на клиент

```
static void handle(Socket client) {
    try (client;
         var in = new BufferedReader(new InputStreamReader(
                 client.getInputStream(), StandardCharsets.UTF_8));
         var out = new PrintWriter(new OutputStreamWriter(
                 client.getOutputStream(), StandardCharsets.UTF_8), true)) {

        String line;
        while ((line = in.readLine()) != null) {     // null = клиентът затвори
            out.println("EXO: " + line);
        }
    } catch (IOException e) {
        System.err.println("Връзката прекъсна: " + e.getMessage());
    }
}
```

`try (client; ...)` – сокетът се затваря автоматично; `true` в `PrintWriter` = автоматичен `flush`

Клиент

```
public class EchoClient {
    public static void main(String[] args) throws IOException {
        try (Socket s = new Socket("localhost", 5000);
             var in = new BufferedReader(new InputStreamReader(
                     s.getInputStream(), StandardCharsets.UTF_8));
             var out = new PrintWriter(new OutputStreamWriter(
                     s.getOutputStream(), StandardCharsets.UTF_8), true);
             var keyboard = new Scanner(System.in)) {

            System.out.println("Свързан. Пишете (празен ред — край):");
            String line;
            while (!(line = keyboard.nextLine()).isEmpty()) {
                out.println(line);                    // към сървъра
                System.out.println(in.readLine());    // отговор
            }
        }
    }
}
```

Как работи връзката?

- Сървърът създава `ServerSocket(5000)` и извиква `accept()` – **блокира**
- Клиентът създава `Socket("localhost", 5000)` – TCP „ръкостискане“
- `accept()` връща нов `Socket` за този клиент
- Двете страни получават **входен** и **изходен** поток
- Обмен на редове текст – протоколът е „ред → отговор“
- `PrintWriter(..., true)` – автоматично изпращане (`flush`) след `println`
- Затваряне на сокета от едната страна → `readLine()` връща `null` при другата

Тестване без собствен клиент

- `nc` (netcat) – macOS, Linux: `nc localhost 5000`
- `telnet localhost 5000` – Windows (след включване)
- Всеки ред, който напишем, се изпраща към сървъра
- Удобно за проверка на текстови протоколи
- Сървърът и клиентът могат да се пуснат в Eclipse като две отделни програми
- Конзолите се превключват от бутона **Display Selected Console**



Проблем: ехо сървърът обслужва само един клиент наведнъж

Докато `while` цикълът чете от първия клиент, `accept()` не се извиква и следващите клиенти чакат. Решение: всеки клиент – в **отделна нишка**.

Многонишков сървър с виртуални нишки

```
public class MultiEchoServer {

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(5000)) {
            while (true) {
                Socket client = server.accept();
                Thread.startVirtualThread(() -> handle(client));  // своя нишка
            }
        }
    }

    // handle(...) — същият метод като при ехо сървъра
}
```

Единствената промяна: `handle` се изпълнява в нова виртуална нишка, а `main` веднага се връща към `accept()`

Проектиране на собствен протокол

- Текстов протокол – лесен за отстраняване на грешки
- Пример: протокол за калкулатор
  - клиент: `ADD 3 5` → сървър: `OK 8`
  - клиент: `DIV 1 0` → сървър: `ERR деление на нула`
  - клиент: `QUIT` → сървърът затваря връзката
- Ясно определено: формат на заявка, формат на отговор, грешки, край
- Сложни данни – JSON на един ред

Сървър с протокол: калкулатор

```
static String process(String request) {
    String[] p = request.strip().split("\\s+");
    try {
        double a = Double.parseDouble(p[1]), b = Double.parseDouble(p[2]);
        return switch (p[0].toUpperCase()) {
            case "ADD" -> "OK " + (a + b);
            case "SUB" -> "OK " + (a - b);
            case "MUL" -> "OK " + (a * b);
            case "DIV" -> b == 0 ? "ERR деление на нула" : "OK " + (a / b);
            default    -> "ERR непозната команда";
        };
    } catch (RuntimeException e) {
        return "ERR невалидна заявка";
    }
}

// в handle(): while ((line = in.readLine()) != null && !line.equals("QUIT"))
//                 out.println(process(line));
```

Клиент за калкулатора

- Изпраща заявки по протокола
- Разпознава `OK` и `ERR` в отговора
- `QUIT` – учтиво затваряне на връзката

```
try (Socket s = new Socket("localhost", 5000);
     var in = new BufferedReader(
         new InputStreamReader(s.getInputStream()));
     var out = new PrintWriter(
         s.getOutputStream(), true)) {

    for (String req : List.of("ADD 3 5",
             "DIV 1 0", "POW 2 3")) {
        out.println(req);
        String resp = in.readLine();
        System.out.println(req + " => " + resp);
    }
    out.println("QUIT");
}
// ADD 3 5 => OK 8.0
// DIV 1 0 => ERR деление на нула
```

Чат сървър: списък с клиенти

```
public class ChatServer {
    // безопасно множество — достъпва се от много нишки
    private static final Set<PrintWriter> clients = ConcurrentHashMap.newKeySet();

    public static void main(String[] args) throws IOException {
        try (ServerSocket server = new ServerSocket(6000)) {
            while (true) {
                Socket s = server.accept();
                Thread.startVirtualThread(() -> handle(s));
            }
        }
    }

    static void broadcast(String msg) {
        for (PrintWriter w : clients) {
            w.println(msg);
        }
    }
}
```

Чат сървър: обслужване на участник

```
static void handle(Socket s) {
    try (s;
         var in = new BufferedReader(
                 new InputStreamReader(s.getInputStream(), UTF_8));
         var out = new PrintWriter(
                 new OutputStreamWriter(s.getOutputStream(), UTF_8), true)) {
        String name = in.readLine();          // първият ред е името
        clients.add(out);
        broadcast("*** " + name + " влезе");

        String msg;
        while ((msg = in.readLine()) != null) {
            broadcast(name + ": " + msg);
        }
        clients.remove(out);
        broadcast("*** " + name + " излезе");
    } catch (IOException e) { /* участникът прекъсна */ }
}
```

Времена за изчакване и грешки

- `setSoTimeout(ms)` – `read` не чака вечно → `SocketTimeoutException`
- `connect(addr, timeout)` – ограничава свързването
- Типични изключения:
  - `ConnectException` – няма сървър на порта
  - `UnknownHostException` – грешно име
  - `SocketException` – прекъсната връзка
  - `BindException` – портът е зает

```
Socket s = new Socket();
s.connect(new InetSocketAddress(
        "localhost", 5000), 3000);
s.setSoTimeout(5000);

try {
    String reply = in.readLine();
} catch (SocketTimeoutException e) {
    System.out.println("Сървърът не отговаря");
}
```

UDP накратко

- `DatagramSocket` – изпраща и получава пакети
- `DatagramPacket` – данни + адрес + порт
- Без връзка, без гаранция за доставка

```
// Получател
try (var sock = new DatagramSocket(6000)) {
    byte[] buf = new byte[1024];
    var p = new DatagramPacket(buf, buf.length);
    sock.receive(p);
    System.out.println(new String(
        p.getData(), 0, p.getLength()));
}

// Изпращач
try (var sock = new DatagramSocket()) {
    byte[] d = "Здравей".getBytes();
    sock.send(new DatagramPacket(d, d.length,
        InetAddress.getLoopbackAddress(), 6000));
}
```

Сигурност на мрежовата комуникация

- Обикновеният TCP предава данните **в открит текст** – всеки по пътя може да ги прочете
- **TLS** – криптиране и проверка на сървъра чрез сертификат
- **HTTPS** = HTTP върху TLS, порт 443
- В Java: `SSLSocket`, а `HttpClient` използва TLS автоматично за `https://`
- Никога не изключвайте проверката на сертификатите „за да работи“
- Проверявайте и ограничавайте входа от мрежата – той идва от непознат

07

HTTP – протокол върху TCP

От сокети към уеб услуги – мост към лекцията за ИИ

HTTP е текст върху TCP

- Клиентът изпраща **заявка**: метод, път, заглавия, празен ред, тяло
- Сървърът връща **отговор**: код на състояние, заглавия, тяло
- Методи: `GET`, `POST`, `PUT`, `DELETE`
- Кодове: 200 OK, 404 Not Found, 500 Internal Server Error

```
GET /hello HTTP/1.1
Host: localhost:8080
Accept: text/plain

HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 8

Здравей!
```

HTTP заявка „на ръка“ със Socket

```
try (Socket s = new Socket("example.com", 80);
     var out = new PrintWriter(s.getOutputStream());
     var in = new BufferedReader(new InputStreamReader(s.getInputStream()))) {

    out.print("GET / HTTP/1.1\r\n");
    out.print("Host: example.com\r\n");
    out.print("Connection: close\r\n");
    out.print("\r\n");                          // празен ред — край на заглавията
    out.flush();

    String line;
    while ((line = in.readLine()) != null) {
        System.out.println(line);               // HTTP/1.1 200 OK ...
    }
}
```

Така работи всеки браузър – само че през HTTPS (TLS) на порт 443

HttpClient (Java 11)

- Стандартен клиент за HTTP/1.1 и HTTP/2
- Поддържа HTTPS, пренасочвания, асинхронни заявки
- `HttpRequest` – строи се с Builder
- `HttpResponse` – код, заглавия, тяло

```
HttpClient client = HttpClient.newHttpClient();

HttpRequest req = HttpRequest.newBuilder()
        .uri(URI.create(
            "https://api.github.com/users/octocat"))
        .header("Accept", "application/json")
        .GET()
        .build();

HttpResponse<String> resp = client.send(req,
        HttpResponse.BodyHandlers.ofString());

System.out.println(resp.statusCode()); // 200
System.out.println(resp.body());       // JSON
```

POST заявка с JSON

- Тялото се подава с `BodyPublishers.ofString`
- Заглавието `Content-Type` указва формата
- Така ще изпращаме заявки към ИИ услуги в лекция 10

```
String json = """
        {"name": "Ана", "course": 2}
        """;

HttpRequest req = HttpRequest.newBuilder()
        .uri(URI.create(
            "https://httpbin.org/post"))
        .header("Content-Type",
                "application/json")
        .POST(HttpRequest.BodyPublishers
                .ofString(json))
        .build();

var resp = client.send(req,
        HttpResponse.BodyHandlers.ofString());
```

Прост HTTP сървър от JDK

```
import com.sun.net.httpserver.HttpServer;

public class HelloServer {
    public static void main(String[] args) throws IOException {
        var server = HttpServer.create(new InetSocketAddress(8080), 0);

        server.createContext("/hello", ex -> {
            byte[] body = "Здравей от Java!".getBytes(UTF_8);
            ex.getResponseHeaders().add("Content-Type", "text/plain; charset=utf-8");
            ex.sendResponseHeaders(200, body.length);
            try (var os = ex.getResponseBody()) { os.write(body); }
        });

        server.setExecutor(Executors.newVirtualThreadPerTaskExecutor());
        server.start();      // http://localhost:8080/hello
    }
}
```

Обобщение

- Нишката е път на изпълнение в процеса; споделя паметта
- `Thread`, `Runnable`, `start`, `join`, `interrupt`; **виртуални нишки** (Java 21)
- Състезания → `synchronized`, атомарни променливи, конкурентни колекции
- `ExecutorService`, `Future`, `CompletableFuture`
- TCP – надеждна връзка; UDP – бързи пакети без гаранция
- `ServerSocket.accept()` + `Socket` + потоци = клиент-сървър
- Всеки клиент – в своя нишка; ясно дефиниран протокол
- HTTP е текстов протокол върху TCP; `HttpClient` за заявки

Въпроси за самопроверка

- Каква е разликата между процес и нишка?
- Защо `value++` не е безопасна операция при няколко нишки?
- С какво виртуалните нишки са по-добри за сървъри?
- Каква е разликата между TCP и UDP? Дайте примери за приложение.
- Какво прави методът `accept()` и защо блокира?
- Защо еднонишковият ехо сървър обслужва само един клиент?
- Каква е структурата на една HTTP заявка?

Въпроси?

Следваща лекция: Тестване и логване

